

Pthread_detach

In taluni casi è opportuno far sì che lo stato di terminazione di un thread ***T*** non venga memorizzato fintanto che un altro thread ***T'*** relativo allo stesso processo attenda per ***T***, ma sia invece cancellato subito dopo la terminazione di ***T***:

```
#include <pthread.h>

int pthread_detach ( pthread_t *tid);
```

- *tid* è l'ID del thread che si vuole distaccare;
- restituisce 0 in caso di successo, un intero positivo – secondo le convenzioni di <sys/errno.h> – in caso di errore.

però, poi non possiamo chiamare pthread_join

Attributi di un thread

- un thread può essere creato in “detached state” (stato sconnesso)
- un thread può bloccare i tentativi di essere cancellato (cancellabilità)
- altri attributi
 - posizione e dimensione dello stack
 - attributi real-time

Gestione Attributi

```
include <pthread.h>
int pthread_attr_init (pthread_attr_t *attr);
int pthread_attr_destroy(pthread_attr_t *attr);
```

- inizializza e distrugge una struttura per gli attributi di un thread. Uso:
 - si alloca una struttura pthread_attr_t (struttura opaca)
 - si chiama pthread_attr_init
 - si modificano gli attributi contenuti nella struttura usando apposite funzioni (vedere dopo)
 - si passa la struttura a pthread_create
 - si distrugge la struttura con pthread_attr_destroy
- restituiscono 0 se OK, un codice d'errore altrimenti

Gestione Attributi

Creazione di un thread in stato dispatched:

```
#include <pthread.h>

int makethread(void * (*fn)(void *), void arg *) {
    int err;
    pthread_t tid;
    pthread_attr_t attr;

    err = pthread_attr_init(&attr);
    iff(err!=0) return (err);
    err =
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED)
    ;
    iff (err==0)
        err = pthread_create(&tid, &attr, fn, arg);
    pthread_attr_destroy(&attr);
    return err;
}
```

Thread-Specific Data

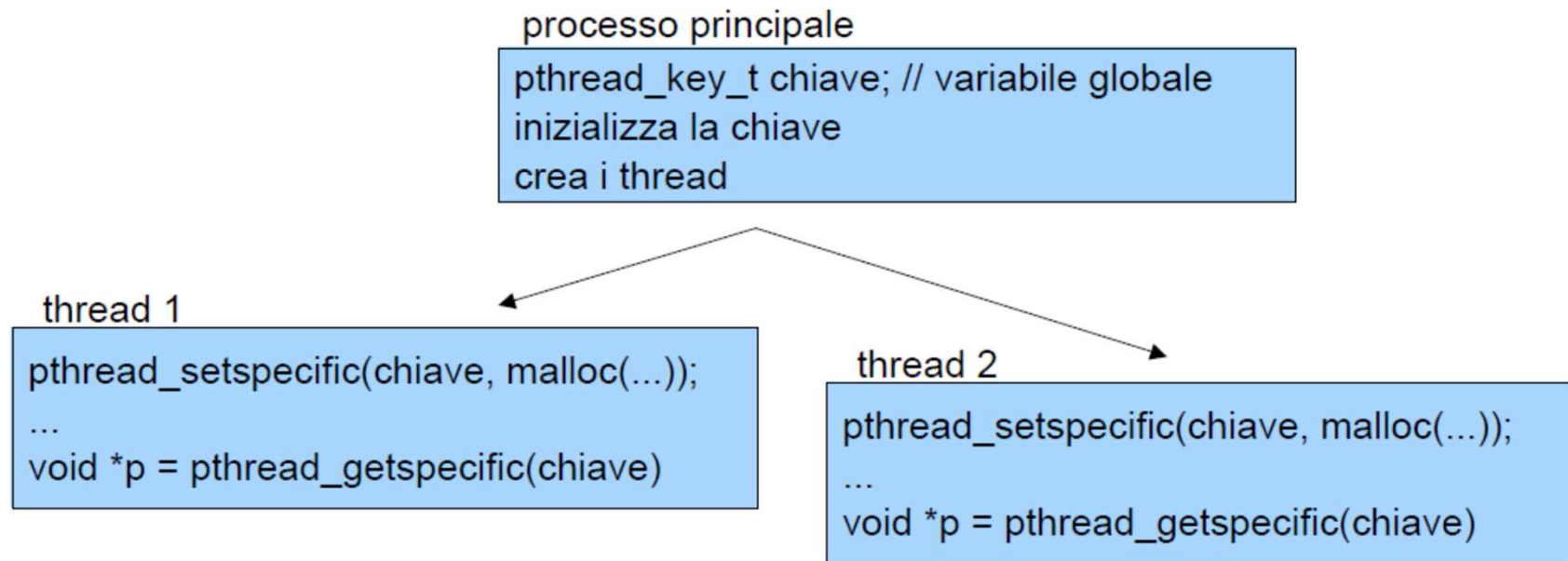
- **Thread-local storage (TLS)** o **Thread-specific Data (TSD)**
 - Nei thread var globali condivise, var locali no
 - Ogni thread può mantenere una sua copia dei dati
 - Utile quando non si controlla direttamente la creazione dei thread (i.e., usando i thread pool)
- Differenza rispetto a variabili locali
 - Variabili locali visibili solo per una singola invocazione di funzione
 - TSD visibile per più invocazioni di funzione
- Simile ai dati **static** (ma senza race condition)
 - TSD unico per ogni thread

Thread-Specific Data (Posix)

- Ogni thread possiede un'area di memoria privata, la TSD area, indicizzata da chiavi
- La TSD area contiene associazioni tra le chiavi ed un valore di tipo void*
 - diversi thread possono usare le stesse chiavi ma i valori associati variano di thread in thread
 - inizialmente tutte le chiavi sono associate a NULL

Thread-Specific Data (Posix)

- associare a una stessa chiave, dati diversi per ciascun thread



Thread-Specific Data (Posix)

- `int pthread_key_create(...)`
 - per creare una chiave TSD
- `int pthread_key_delete(...)`
 - per deallocare una chiave TSD
- `int pthread_setspecific(...)`
 - per associare un certo valore ad una chiave TSD
- `void * pthread_getspecific(...)`
 - per ottenere il valore associato ad una chiave TSD

Thread-Specific Data (Posix)

```
int pthread_key_create(pthread_key_t *key,  
                        void (*destructor)(void *));
```

- crea una chiave per dati privati
- key è l'indirizzo della chiave da inizializzare
- destructor è un puntatore alla funzione distruttore che deve essere chiamata alla terminazione di un thread (pthread_exit())
- restituisce 0 se OK, un codice d'errore altrimenti

Thread-Specific Data (Posix)

```
int pthread_setspecific(pthread_key_t *key,  
                        const void* val);
```

- associa l'indirizzo val alla chiave key, per il thread chiamante
 - restituisce 0 se OK, un codice d'errore altrimenti
-

```
void* pthread_getspecific(pthread_key_t *key);
```

- restituisce l'indirizzo associato alla chiave key nel thread chiamante
 - restituisce NULL se nessun indirizzo è stato associato a key

Thread-Specific Data (Posix)

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

/* Chiave globale per i dati specifici del thread */
pthread_key_t chiave;

/* Funzione eseguita da ogni thread */
void *worker(void *arg) {
    int *id = (int *)arg; // id passato dal main
    int *val = malloc(sizeof(int));
    *val = *id * 10;      // calcolo un valore solo per
    questo thread

    /* associo il valore alla chiave (specifico per thread)
    */
    pthread_setspecific(chiave, val);

    /* recupero il valore dalla chiave e lo stampo */
    int *mio_val = pthread_getspecific(chiave);
    printf("[thread %d] valore specifico = %d\n", *id,
    *mio_val);

    free(val); // libero la memoria
    return NULL;
}
```

```
int main(void) {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    /* creo la chiave globale */
    pthread_key_create(&chiave, NULL);

    /* creo due thread, ciascuno con un proprio id */
    pthread_create(&t1, NULL, worker, &id1);
    pthread_create(&t2, NULL, worker, &id2);

    /* attendo la fine dei thread */
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    /* elimino la chiave */
    pthread_key_delete(chiave);

    return 0;
}
```

Thread-Specific Data (Posix)

```
#include ...

static pthread_key_t thread_log_key; /* tsd key per thread */

void write_to_thread_log (const char* message); //Scrive log
void close_thread_log (void* thread_log); //Chiude file log
void* thread_function (void* args);           //Eseguita dai thread

int main() {
    // Crea una chiave da associare al log Thread-Specific
    // Crea 5 thread che facciano il lavoro
    // Aspetta che tutti finiscano
    return 0;
}
...
```

Thread-Specific Data (Posix)

```
...
int main() {
    int i;
    pthread_t threads[5];
    // Crea una chiave da associare al puntatore TSD al log file
    pthread_key_create(&thread_log_key, close_thread_log);

    for (i = 0; i < 5; ++i) // thread che faccia il lavoro
        pthread_create(&(threads[i]), NULL, thread_function, NULL);

    for (i = 0; i < 5; ++i) // Aspetta che tutti finiscano
        pthread_join (threads[i], NULL);
    return 0;
}
```

Thread-Specific Data (Posix)

```
...
void write_to_thread_log (const char* message) {
    FILE* thread_log = (FILE*)pthread_getspecific(thread_log_key);
    fprintf (thread_log, "%s\n", message);
}

void close_thread_log (void* thread_log) {
    fclose ((FILE*) thread_log);
}

void* thread_function (void* args) {
    char thread_log_filename[20];
    FILE* thread_log;
    sprintf(thread_log_filename, "thread%d.log", (int)pthread_self ());

    thread_log = fopen (thread_log_filename, "w");
    /* Associa la struttura FILE TSD a thread_log_key. */
    pthread_setspecific (thread_log_key, thread_log);

    write_to_thread_log ("Thread starting.");
    /* Fai altro lavoro qui... */ return NULL;
}
```


Task in Linux

- Linux si riferisce a **tasks** piuttosto che a **threads**
 - Unificando la gestione di processi e thread
- Thread creation è gestita dalla chiamata di sistema **clone()**
 - Usata per implementare **pthread_create**
- **clone()** permette ad un child task di condividere l'address space del parent task (processo)
 - Flag di controllo:

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

Viene comunque
usata la chiamata a
`fork()`:
`do_fork()`

- Se tutte settate allora clone come thread, se tutte non settate come fork
- **struct task_struct** punta alle strutture dati del processo (condivise o uniche) - gestione uniforme

Thread Linux

□ Esempio di uso del `clone()`

```
int thread_func(void *arg) {
    printf("Hello from the new thread! Arg: %s\n", (char *)arg);
    return 0;
}

int main() {
    char *child_stack = malloc(1024 * 1024); // Allocazione dello stack
    if (child_stack == NULL) {
        perror("malloc");
        exit(1);
    }

    // Creazione del nuovo thread con clone()
    pid_t pid = clone(thread_func, child_stack + 1024 * 1024, CLONE_VM | CLONE_FS | CLONE_FILES |
        CLONE_SIGHAND | CLONE_THREAD, "Thread Arg");

    if (pid == -1) {
        perror("clone");
        exit(1);
    }

    printf("Thread created with pid: %d\n", pid);
    sleep(1); // Aspetta che il thread finisca
    free(child_stack); // Libera lo stack

    return 0;
}
```

Thread Linux

- ❑ Comandi per vedere i Kernel thread: lightweight Processes (LWP)
 - ❑ `ps -Lf <PID>`
 - ❑ `ps -T -p <PID>`
 - ❑ `ls /proc/<PID>/task`
- ❑ Differenza tra tid da libreria (`pthread_self()`) e tid LWP
 - ▶ `pthread_t` tipo opaco
 - ▶ SPID intero come il PID

Thread Linux

```
int main(void) {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    printf("[main] PID (Thread Group ID) = %d\n", getpid());

    pthread_create(&t1, NULL, worker, (void *)&id1);
    pthread_create(&t2, NULL, worker, (void *)&id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Thread Linux

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/syscall.h>    // per SYS_gettid
#include <stdint.h>

void *worker(void *arg) {
    int id = *(int *)arg;
    char c;

    pthread_t self = pthread_self();        // ID POSIX
    pid_t tid = syscall(SYS_gettid);
    // ID del kernel (TID reale)

    printf("[thread %d] pthread_self() = %lu, gettid() = %d,
PID (TGID) = %d\n",
        id, (unsigned long)self, tid, getpid());

    /* blocco per osservare i thread con ps */
    read(STDIN_FILENO, &c, 1);

    return NULL;
}
```

Riferimenti

- Advanced Programming in the Unix Environment (Second Ed.)
 - Threads: 11.1, 11.2, 11.3, 11.4, 11.5

Sincronizzazione

Sincronizzazione di thread POSIX

Sincronizzazione

- I thread condividono la memoria
- Rischio di race condition, se accesso di più thread a stessi dati (e.g. lettura/scrittura di stesse variabili)
- Necessari meccanismi di sincronizzazione
 - mutex (semaforo binario)
 - condition variable

Sincronizzazione

- I thread condividono la memoria
- Rischio di race condition, se accesso di più thread a stessi dati (e.g. lettura/scrittura di stesse variabili)
- Necessari meccanismi di sincronizzazione
 - mutex (semaforo binario)
 - condition variable

Sincronizzazione

- La sincronizzazione e' necessaria quando si accede a variabili condivise
 - Variabili/Strutture dati globali (statiche e dinamiche)
- Attenzione: TUTTE le operazioni possono essere NON atomiche
 - Dipende dall'architettura (se più accessi in mem. per una operazione, un thread si può inserire durante).
- E.g. “x++” puo' diventare:
 - Carica la variabile x in accumulatore
 - Incrementa l'accumulatore
 - Memorizza l'accumulatore

Corsa Critica

- Operazioni non atomiche
- Se i due processi eseguono `counter++` e `counter--` insieme
- `counter++` può essere implementato come

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- `counter--` può essere implementato come

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Interfogliate le due esecuzioni, inizialmente “count = 5”:

S0: producer	<code>register1 = counter</code>	{register1 = 5}
S1: producer	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer	<code>register2 = counter</code>	{register2 = 5}
S3: consumer	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer	<code>counter = register1</code>	{counter = 6}
S5: consumer	<code>counter = register2</code>	{counter = 4}

Istruzioni atomiche

- Macchine moderne forniscono hardware per istruzioni atomiche
 - Atomiche = non-interrompibili
 - Eseguibili sequenzialmente
- Esempi:
 - Test memory (word) and set (value)
 - Swap contents of two memory words

compare_and_swap

Definizione:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

1. Eseguite atomicamente
2. Restituisce il valore originale del parametro in “value”
3. Setta la variabile “value” al valore di “new_value” ma solo se “value” == “expected”. Cioè lo swap avviene solo con questa condizione.

Su architetture Intel x86 istruzione per CAS con lock del bus

```
lock cmpxchg <destination operand>, <source operand>
```

Comparazione tra acc e dest, se uguale dest = src, altrimenti acc = dest

Corsa Critica

- Il problema della corsa critica è pervasivo
- In particolare in sistemi multicore dove multithreading è molto enfatizzato
- Occorrono meccanismi di sincronizzazione e coordinazione dei processi

Esempio

```
typedef struct foo{ int a;  int b; } myfoo;

myfoo test; // Variabile GLOBALE

void *inc(void *arg){ // incremente a e b
    test.a++;
    test.b++;
    printf("tid=%d a=%d b=%d\n" , pthread_self(), test.a, test.b);
    pthread_exit((void *)&test);
}

int main(void){
    char st[100];
    pthread_t tid;
    int i=0;
    myfoo *b;

    while (i++<10){
        pthread_create(&tid, NULL, inc, NULL); // Thread concorrenti
    }
    sleep(1);
}
```

Esempio: race

tid=1077283760 a=1 b=1
tid=1089891248 a=5 b=5
tid=1079385008 a=6 b=6
tid=1081486256 a=7 b=7
tid=1083587504 a=8 b=8
tid=1085688752 a=9 b=9
tid=1087790000 a=10 b=10
tid=1096194992 a=2 b=2
tid=1094093744 a=3 b=3
tid=1091992496 a=4 b=4

Esempio: race

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
int myglobal;
```

```
void *thread_function(void *arg) {
    int i, j;
```

```
    for (i = 0; i < 20; i++) {
        j = myglobal;
        j = j + 1;
        printf(".");
        fflush(stdout);
        sleep(1);
        myglobal = j;
    }
```

```
    return NULL;
}
```

```
int main(void) {
    pthread_t mythread;
    int i;

    if (pthread_create(&mythread, NULL, thread_function, NULL)) {
        printf("Error creating thread\n");
        abort();
    }

    for (i = 0; i < 20; i++) {
        myglobal = myglobal + 1;
        printf("o");
        fflush(stdout);
        sleep(1);
    }

    if (pthread_join(mythread, NULL)) {
        printf("Error joining thread\n");
        abort();
    }

    printf("\nmyglobal equals %d\n", myglobal);
    return 0;
}
```

Esempio: race

Esecuzione:

```
$ ./race
```

Possibile Output:

```
...o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o.o
```

```
myglobal è uguale a 21
```

Problema della Sezione Critica

- Considera un sistema di n processi $\{p_0, p_1, \dots, p_{n-1}\}$
- Ogni processo ha un segmento di **sezione critica** nel codice
 - Processi potrebbero cambiare variabili comuni, aggiornare tabelle condivise, scrivere file, etc.
 - Quando un processo entra in sezione critica gli altri processi non dovrebbero andare nella loro sezione critica
- **Problema della sezione critica** richiede un protocollo di interazione per risolverlo
 - Ogni processo deve chiedere il permesso per entrare nella sezione critica.
 - Il codice che implemente la richiesta è la **entry section**
 - Dopo la sezione ci sarà una **exit section**
 - Il codice rimanente che segue è la **remainder section**

Critical Section

- Struttura generale di un processo con sezione critica P_i

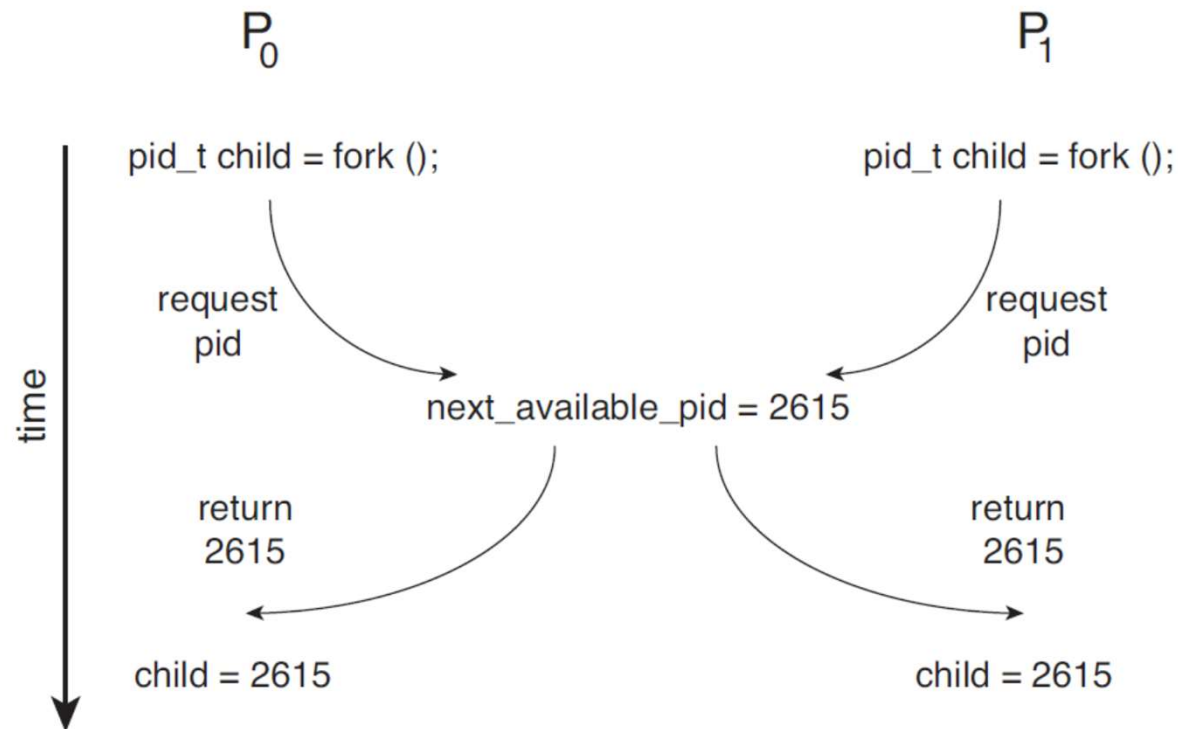
```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Problema della Sezione Critica

1. **Mutua Esclusione** - se il processo P_i esegue la sua sezione critica nessun altro processo può eseguire la sua sezione critica
2. **Progresso** - Se nessun processo è in esecuzione nella sua sezione critica ed esistono alcuni processi che desiderano entrare nella loro sezione critica, solo i processi non in remainder section decideranno la selezione dei processi che entreranno successivamente nella sezione critica, che non può essere posticipata indefinitamente
3. **Bounded Waiting** - Deve esistere un limite al numero di volte in cui altri processi possono entrare nelle loro sezioni critiche dopo che un processo ha fatto una richiesta di entrare nella sua sezione critica prima che tale richiesta sia concessa
 - Assunto che ogni processo esegue a velocità non zero
 - Senza assunzioni sulla **velocità relativa** degli n processi

Corse Critiche nel Kernel

- I processi P_0 e P_1 creano processi figlio utilizzando `fork()`
- Corse critiche su tabella di file aperti nel sistema
- Corse critiche sulla variabile kernel che rappresenta il prossimo identificatore di processo disponibile (`pid`), `next_available_pid`
 - Senza mutua esclusione, lo stesso pid potrebbe essere assegnato a due processi diversi



Mutex Posix

Un mutex Posix è caratterizzato dalle seguenti proprietà:

- è una variabile di tipo `pthread_mutex_t` che può essere inizializzata con diversi attributi (`tipo`, `scopo`, etc.)
- può assumere solo i due stati alternativi `chiuso` (locked) o `aperto` (unlocked);
- può essere chiuso solo da `un` processo alla volta, ed il processo che chiude il mutex ne diviene il `possessore` fino alla successiva chiusura;
- può essere riaperto solo dal proprio `possessore`;
- deve essere `condiviso` tra tutti i processi che intendono sincronizzare l'accesso ad una regione critica (`blocco cooperativo`).

Mutex Posix

- Un mutex è un semaforo binario (rosso o verde)
- Un mutex è mantenuto in una struttura

`pthread_mutex_t`

- Tale struttura va allocata e inizializzata
- Per inizializzare:

- se la struttura è allocata staticamente:

```
pthread_mutex_t c = PTHREAD_MUTEX_INITIALIZER
```

- se la struttura è allocata dinamicamente (e.g. se si usa malloc): chiamare `pthread_mutex_init`

Mutex Posix

```
#include <pthread.h>
int pthread_mutex_init(          pthread_mutex_t  *mutex,
                           const pthread_mutexattr_t *attr);

int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- inizializza e distrugge un mutex, rispettivamente
- Quando inizializzato e' in stato aperto
- restituiscono 0 se OK, un codice d'errore altrimenti
- attr può essere NULL (attributi di default)

Usare i Mutex

```
int pthread_mutex_lock      (pthread_mutex_t *mutex);  
int pthread_mutex_trylock  (pthread_mutex_t *mutex);  
int pthread_mutex_unlock  (pthread_mutex_t *mutex);
```

- acquisiscono e rilasciano il semaforo
- restituiscono 0 se OK, un codice d'errore altrimenti
- se il semaforo è occupato (locked)...
 - ...lock blocca il thread finché il semaforo si libera
 - ...trylock invece non blocca, ma restituisce subito l'errore EBUSY

Esempio

```
myfoo test; // Variabile GLOBALE
pthread_mutex_t sem=PTHREAD_MUTEX_INITIALIZER;

void *inc(void *arg){ // incremente a e b
    pthread_mutex_lock(&sem);
    test.a++;
    test.b++;
    printf("tid=%d a=%d b=%d\n" , pthread_self(), test.a, test.b);
    pthread_mutex_unlock(&sem);
    pthread_exit((void *)&test);
}

int main(void){
    char st[100];
    pthread_t tid;
    int i=0;
    myfoo *b;

    while (i++<10){
        pthread_create(&tid, NULL, inc, NULL); // Thread concorrenti
    }
```

Esempio

tid=1077283760 a=1 b=1

tid=1096194992 a=2 b=2

tid=1079385008 a=3 b=3

tid=1081486256 a=4 b=4

tid=1083587504 a=5 b=5

tid=1085688752 a=6 b=6

tid=1087790000 a=7 b=7

tid=1094093744 a=8 b=8

tid=1091992496 a=9 b=9

tid=1089891248 a=10 b=10

Esempio

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int myglobal = 0;
pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;

void *thread_function(void *arg) {
    int i;

    for (i = 0; i < 20; i++) {
        pthread_mutex_lock(&mymutex);
        myglobal = myglobal + 1;
        pthread_mutex_unlock(&mymutex);

        printf(".");
        fflush(stdout);
        sleep(1);
    }

    return NULL;
}
```

Esempio

```
int main(void) {
    pthread_t mythread;
    int i;

    if (pthread_create(&mythread, NULL, thread_function, NULL)) {
        printf("Errore nella creazione del thread\n");
        exit(1);
    }

    for (i = 0; i < 20; i++) {
        pthread_mutex_lock(&mymutex);
        myglobal = myglobal + 1;
        pthread_mutex_unlock(&mymutex);

        printf("o");
        fflush(stdout);
        sleep(1);
    }

    if (pthread_join(mythread, NULL)) {
        printf("Errore nel join del thread\n");
        exit(2);
    }

    printf("\nmyglobal è uguale a %d\n", myglobal);
    return 0;
}
```

Esecuzione:

\$./race

Possibile Output:

\$./race

```
0..0..0.0..0..0.0.0.0.0..0..0..0.00000000
```

myGlobal è uguale a 40

Sincronizzazione

- La sincronizzazione puo' essere:
 - Per sezione critica
 - SOLO quando una struttura condivisa viene modificata in UN UNICO punto nel codice (esempio precedente)
 - E' sufficiente associare un mutex alla sezione critica
 - Per “struttura”
 - Quando la struttura puo' essere modificata in piu' punti nel codice
 - Utile se piu' strutture devono essere condivise contemporaneamente
 - E' necessario associare un mutex alla “struttura”

Sincronizzazione

- La sincronizzazione puo' essere:
 - Per sezione critica
 - SOLO quando una struttura condivisa viene modificata in UN UNICO punto nel codice (esempio precedente)
 - E' sufficiente associare un mutex alla sezione critica
 - Per “struttura”
 - Quando la struttura puo' essere modificata in piu' punti nel codice
 - Utile se piu' strutture devono essere condivise contemporaneamente
 - E' necessario associare un mutex alla “struttura”

Esempio

```
typedef struct foo{  
    int a;  
    int b;  
    pthread_mutex_t sem;  
} myfoo;
```

```
myfoo *test; // Variabile GLOBALE
```

```
myfoo *init_struct(){  
    struct foo *fp;  
  
    if ((fp=malloc(sizeof(myfoo)))==NULL)  
        return(NULL);  
    fp->a=0;  
    fp->b=0;  
    pthread_mutex_init(&fp->sem,NULL);  
    return(fp);  
}
```

Esempio

```
void stampa(struct foo *test){
    printf("tid=%d a=%d b=%d\n", pthread_self(), test->a, test->b);
    fflush(stdout);
}

void *inc(void *arg){ // incremente a e b
    pthread_mutex_lock(&test->sem);
    test->a=test->a+2;
    test->b++; // Variabile GLOBALE
    stampa(test);
    pthread_mutex_unlock(&test->sem);
    pthread_exit((void *)&test);
}
```

Esempio

```
int main(void){
    char st[100];
    pthread_t tid;
    int i=0;
    myfoo *b;

    test=init_struct();
    while (i++<10){
        pthread_create(&tid, NULL, inc, NULL); // Globale
    }
    sleep(1);
    printf("Master:");
    stampa(test);
    pthread_mutex_destroy(&test->sem);
}
```

Esempio

tid=1077283760 a=2 b=1
tid=1079385008 a=4 b=2
tid=1081486256 a=6 b=3
tid=1083587504 a=8 b=4
tid=1085688752 a=10 b=5
tid=1087790000 a=12 b=6
tid=1089891248 a=14 b=7
tid=1091992496 a=16 b=8
tid=1094093744 a=18 b=9
tid=1096194992 a=20 b=10
Master:tid=1075181248 a=20 b=10

Attributi di un Mutex

```
#include <pthread.h>
```

```
int pthread_mutexattr_init(pthread_mutexattr_t *attr);
```

- crea in `attr` un attributo di mutex come quelli richiesti dalla `pthread_mutex_init()`

```
int pthread_mutexattr_destroy(pthread_mutexattr_t *attr);
```

- dealloca l'attributo di mutex in `attr`;
- Entrambe restituiscono sempre 0
- Attualmente LinuxThreads supporta solo l'attributo relativo al tipo di mutex

Tipologie Mutex

- fast: semantica classica, pertanto un thread che esegue due `mutex_lock()` consecutivi sullo stesso mutex causa uno stallo
- recursive: conta il numero di volte che un thread blocca il mutex e lo rilascia solo se esegue un pari numero di `mutex_unlock()`
- error-checking: controlla che il thread che rilascia il mutex sia lo stesso thread che lo possiede

Tipologie Mutex

- **inizializzazione di un mutex**

- **statica, macro per inizializzare un mutex:**

```
fastmutex    = PTHREAD_MUTEX_INITIALIZER;  
recmutex     = PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP;  
errchkmutex  = PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP;
```

- **dinamica, chiamata di libreria:**

```
int pthread_mutex_init(pthread_mutex_t *mp,  
                        const pthread_mutexattr_t *mattr);
```

- **mp** è una mutex precedentemente allocato
 - **mattr** sono gli attributi del mutex: `NULL` per il default
 - restituisce sempre 0

Tipologie Mutex

- **inizializzazione di un mutex**

- **statica, macro per inizializzare un mutex:**

```
fastmutex    = PTHREAD_MUTEX_INITIALIZER;  
recmutex     = PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP;  
errchkmutex  = PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP;
```

- **dinamica, chiamata di libreria:**

```
int pthread_mutex_init(pthread_mutex_t *mp,  
                        const pthread_mutexattr_t *mattr);
```

- **mp** è una mutex precedentemente allocato
 - **mattr** sono gli attributi del mutex: `NULL` per il default
 - restituisce sempre 0

Lock per Tipologia

■ Lock(): blocca un mutex

- se era sbloccato il thread chiamante ne prende possesso bloccandolo immediatamente e la funzione ritorna subito
- se era bloccato da un altro thread il thread chiamante viene sospeso sino a quando il possessore non lo rilascia
- se era bloccato dallo stesso thread chiamante dipende dal tipo mutex
 - fast: stallo, perché il chiamante stesso, che possiede il mutex, viene sospeso in attesa di un rilascio che non avverrà mai
 - error checking: la chiamata fallisce
 - recursive: la chiamata ha successo, ritorna subito, incrementa il contatore del numero di lock eseguiti dal thread chiamante

Unlock per Tipologia

- `unlock()`: sblocca un mutex che si assume fosse bloccato. Ad ogni modo la semantica esatta dipende dal tipo di mutex
 - `fast`: il mutex viene lasciato sbloccato e la chiamata ha sempre successo
 - `recursive`: si decrementa il contatore del numero di lock eseguiti dal thread chiamante sul mutex, e lo si sblocca solamente se tale contatore si azzerava
 - `error checking`: sblocca il mutex solo se al momento della chiamata era bloccato e posseduto dal thread chiamante, in tutti gli altri casi la chiamata fallisce senza alcun effetto sul mutex

Attributi di un Mutex

- Si può scegliere il tipo usando queste macro:

- fast: `PTHREAD_MUTEX_FAST_NP`
- recursive: `PTHREAD_MUTEX_RECURSIVE_NP`
- error checking: `PTHREAD_MUTEX_ERRORCHECK_NP`

e le funzioni per fissare/conoscere il tipo:

```
#include <pthread.h>
```

```
int pthread_mutexattr_settype(pthread_mutexattr_t *attr,  
                             int kind);
```

- restituisce 0 oppure un intero $\neq 0$ in caso di errore

```
int pthread_mutexattr_gettype(const pthread_mutexattr_t *attr,  
                             int *kind);
```

- restituisce sempre 0

Unlock per Tipologia

- `unlock()`: sblocca un mutex che si assume fosse bloccato. Ad ogni modo la semantica esatta dipende dal tipo di mutex
 - `fast`: il mutex viene lasciato sbloccato e la chiamata ha sempre successo
 - `recursive`: si decrementa il contatore del numero di lock eseguiti dal thread chiamante sul mutex, e lo si sblocca solamente se tale contatore si azzerava
 - `error checking`: sblocca il mutex solo se al momento della chiamata era bloccato e posseduto dal thread chiamante, in tutti gli altri casi la chiamata fallisce senza alcun effetto sul mutex

Deadlock

- Condizione di attesa ciclica
- Soluzione base: acquisire i mutex sempre nello stesso ordine
 - Non sempre possibile!
 - Può essere necessario utilizzare algoritmi specifici e `pthread_mutex_trylock`